

OBJECT ORIENTED PROGRAMMING USING C++

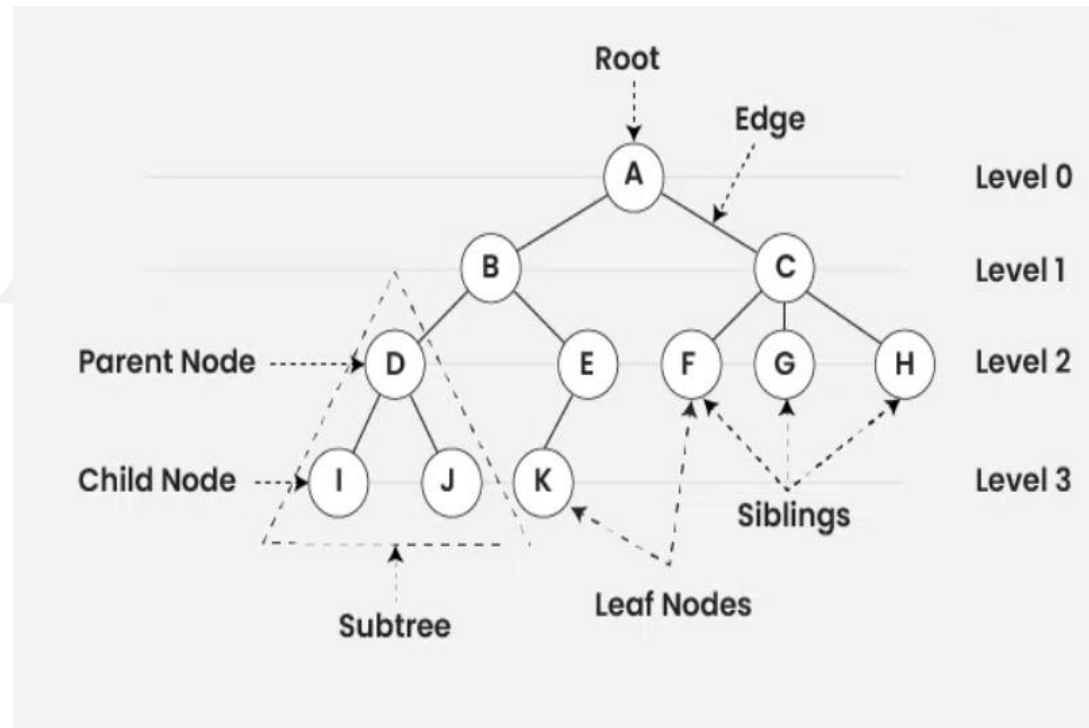


Study Materials

Tree:

- Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search.
- It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.
- The topmost node of the tree is called the root, and the nodes below it are called the child nodes.
- Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

Representation of Trees:



Types of Tree:

- Tree data structure can be classified into three types based upon the number of children each node of the tree can have. The types are:
- Binary tree: In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees. Examples of Binary Tree are Binary Search Tree and Binary Heap.
- Ternary Tree: A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- N-ary Tree or Generic Tree: Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

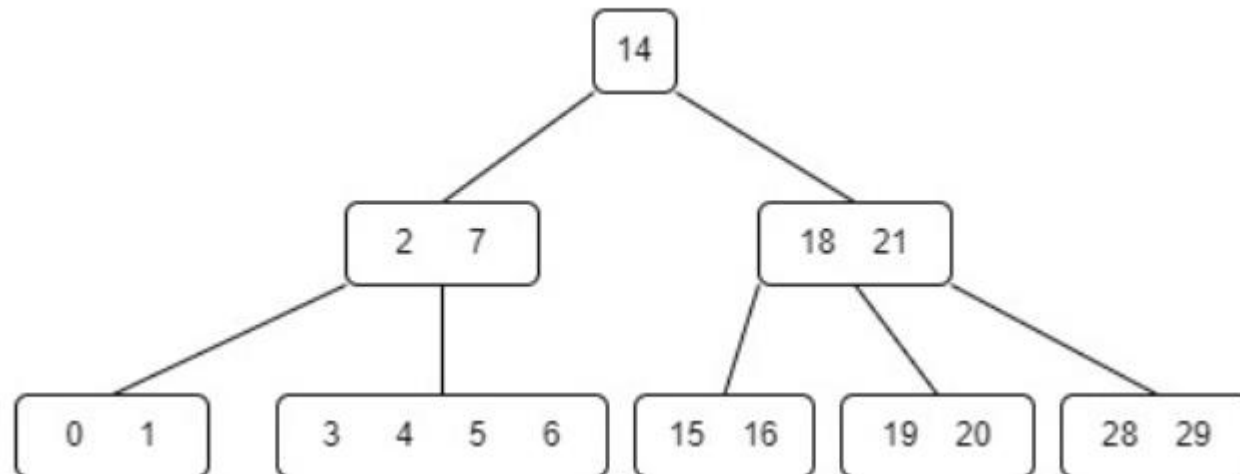
B trees:

- B trees are extended binary search trees that are specialized in m-way searching, since the order of B trees is 'm'.
- Order of a tree is defined as the maximum number of children a node can accommodate.
- Therefore, the height of a b tree is relatively smaller than the height of AVL tree and RB tree.
- They are general form of a Binary Search Tree as it holds more than one key and two children.

Properties of B trees:

- Every node in a B Tree will hold a maximum of m children and $(m-1)$ keys, since the order of the tree is m .
- Every node in a B tree, except root and leaf, can hold at least $m/2$ children
- The root node must have no less than two children.
- All the paths in a B tree must end at the same level, the leaf nodes must be at the same level.
- A B tree always maintains sorted data.
- B trees are also widely used in disk access, minimizing the disk access time since the height of a b tree is low.

Representation of B-trees:



Operations of B Trees:

- The operations supported in B trees are,
- Insertion, deletion and searching with the time complexity of $O(\log n)$ for every operation.



Insertion operation:

- The insertion operation for a B Tree is done similar to the Binary Search Tree but the elements are inserted into the same node until the maximum keys are reached.
- The insertion is done using the following procedure

Insertion operation:

- **Step 1** – Calculate the maximum $(m-1)$ and, minimum $(\lceil m/2 \rceil - 1)$ number of keys a node can hold, where m is denoted by the order of the B Tree.

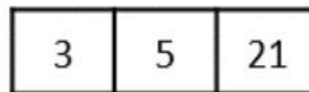
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

- Order $(m) = 4$
- Maximum Keys $(m - 1) = 3$
- Minimum Keys $(\lceil \frac{m}{2} \rceil) - 1 = 1$
- Maximum Children $= 4$
- Minimum Children $(\lceil \frac{m}{2} \rceil) = 2$

Insertion operation:

- **Step 2** – The data is inserted into the tree using the binary search insertion and once the keys reach the maximum number, the node is split into half and the median key becomes the internal node while the left and right keys become its children.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

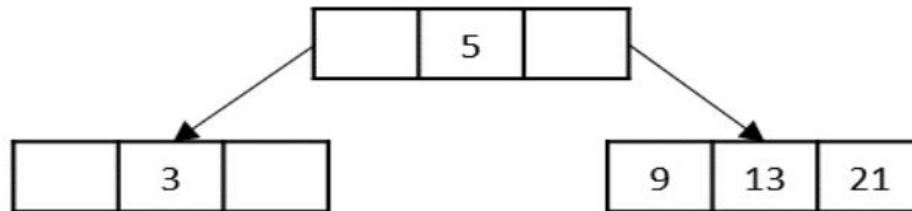


Adding 9 will cause overflow in the node; hence it must be split.

Insertion operation:

- **Step 3** – All the leaf nodes must be on the same level.
- The final B tree after inserting all the elements is achieved.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

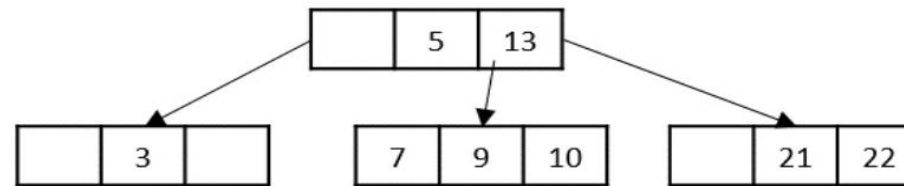


Adding 22 will cause overflow in the node; hence it must be split.

Insertion operation:

- The keys, 5, 3, 21, 9, 13 are all added into the node according to the binary search property but if we add the key 22, it will violate the maximum key property. Hence, the node is split in half, the median key is shifted to the parent node and the insertion is then continued.

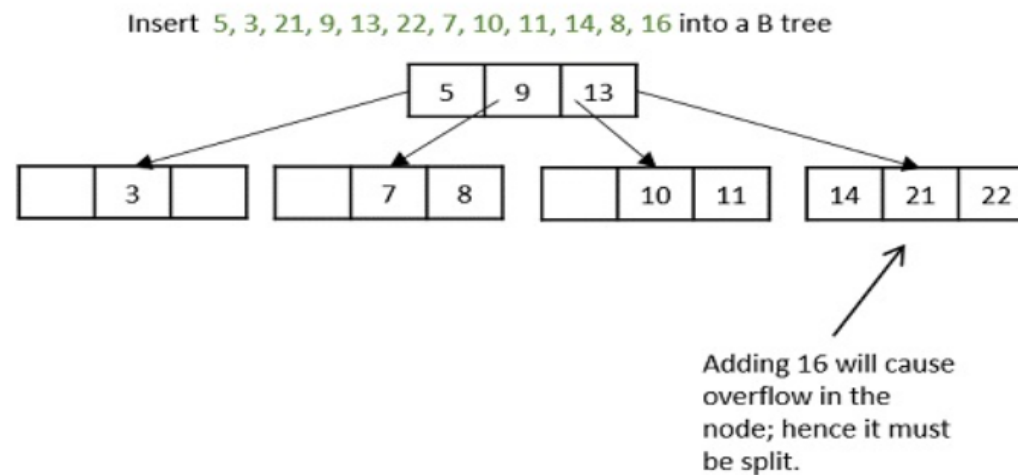
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 11 will cause overflow in the node; hence it must be split.

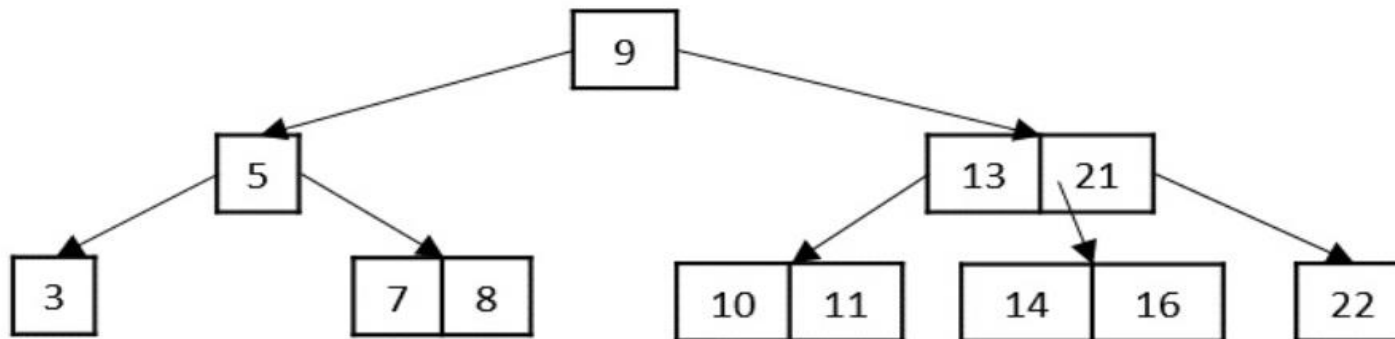
Insertion operation:

- Another hiccup occurs during the insertion of 11, so the node is split and median is shifted to the parent.



Insertion operation:

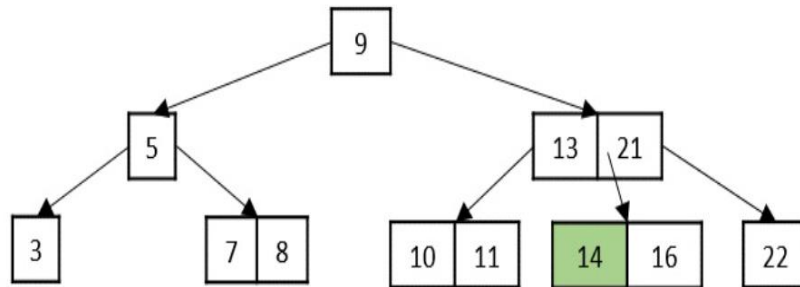
- While inserting 16, even if the node is split in two parts, the parent node also overflows as it reached the maximum keys.
- Hence, the parent node is split first and the median key becomes the root. Then, the leaf node is split in half the median of leaf node is shifted to its parent.



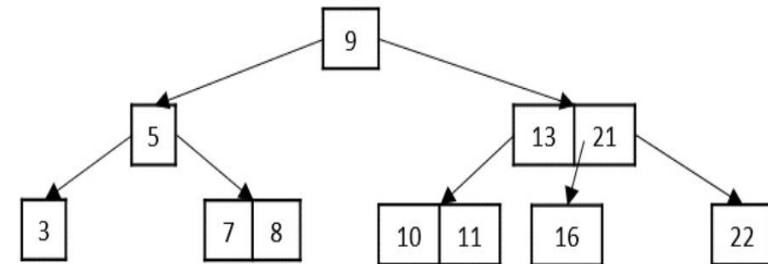
Deletion operation:

- The deletion operation in a B tree is slightly different from the deletion operation of a Binary Search Tree.
- Case 1 – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

Delete key 14



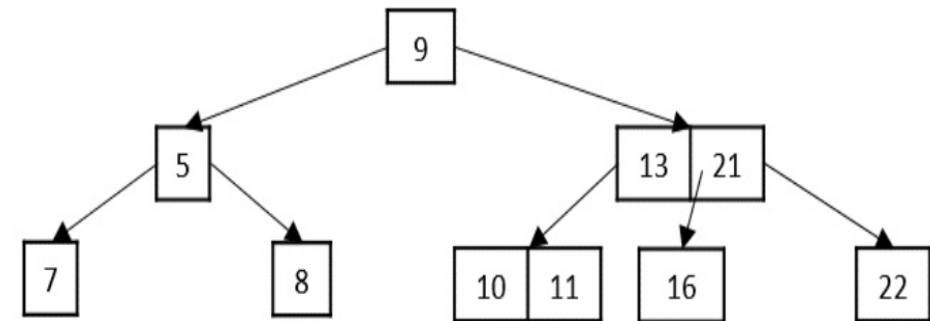
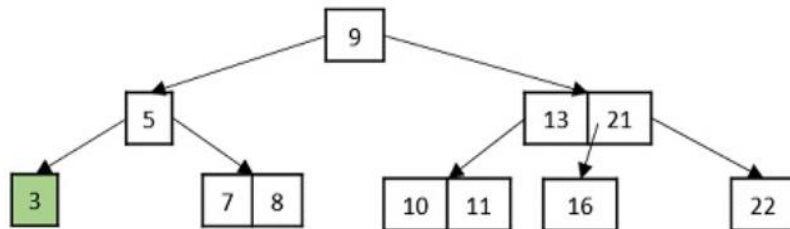
Delete key 14



Deletion operation:

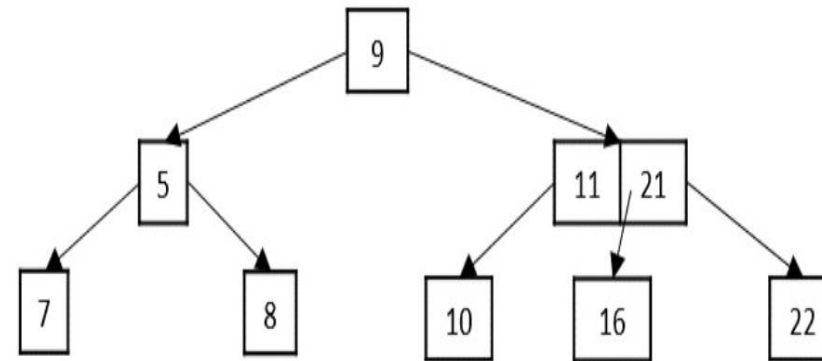
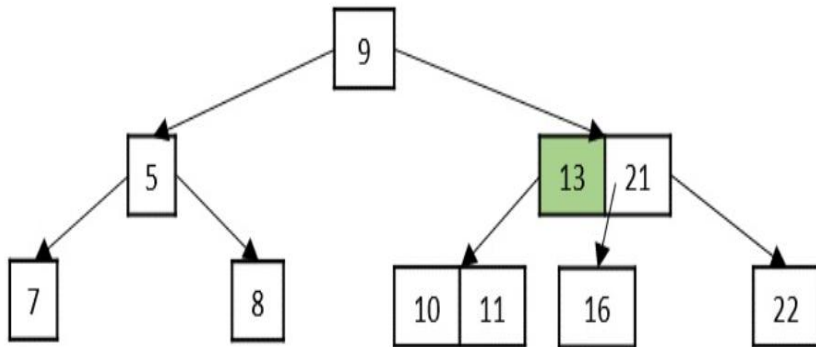
- Case 2 – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling.
- In case if both siblings have exact minimum number of keys, merge the node in either of them.

Delete key 3



Deletion operation:

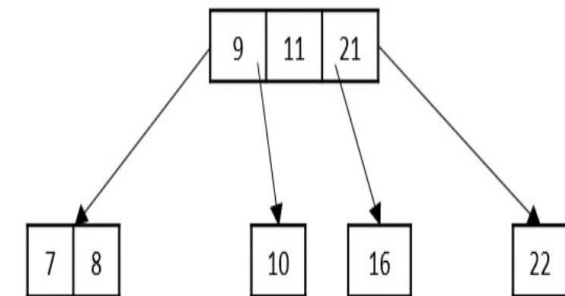
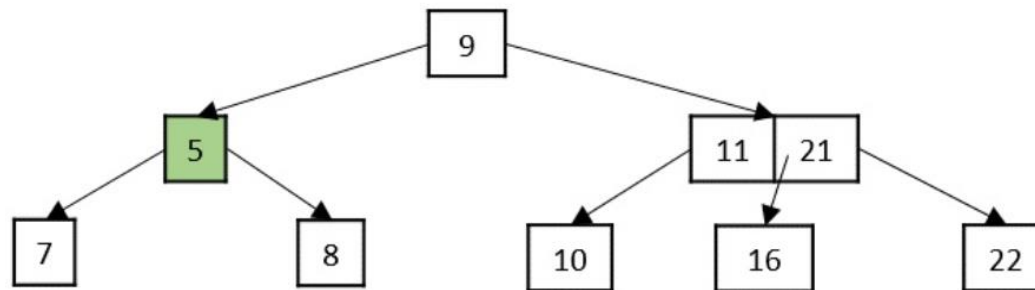
- **Case 3** – If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys.
- But if both child nodes have minimum number of keys, they're merged together.



Deletion operation:

- Case 4 – If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children.
- Then merge its sibling with its parent.

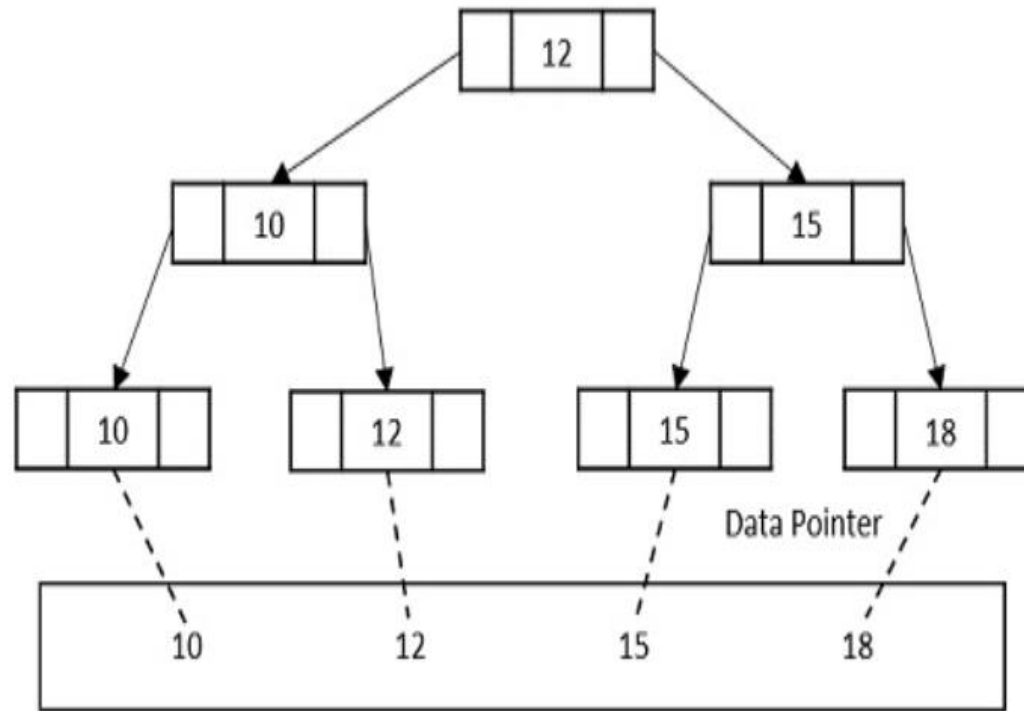
Delete key 5



B + Tree:

- B + Tree is a variation of the B-tree data structure. In a B + tree, data pointers are stored only at the leaf nodes of the tree.
- In a B+ tree structure of a leaf node differs from the structure of internal nodes.
- The leaf nodes have an entry for every value of the search field, along with a data pointer to the record (or to the block that contains this record).
- The leaf nodes of the B+ tree are linked together to provide ordered access to the search field to the records. Internal nodes of a B+ tree are used to guide the search.
- Some search field values from the leaf nodes are repeated in the internal nodes of the B+ tree.

B + Tree:



Properties of B+ trees:

- Every node in a B+ Tree, except root, will hold a maximum of m children and $(m-1)$ keys, and a minimum of $\lceil m/2 \rceil$ children and $\lceil (m-1)/2 \rceil$ keys, since the order of the tree is m .
- The root node must have no less than two children and at least one search key.
- All the paths in a B tree must end at the same level, the leaf nodes must be at the same level.
- A B+ tree always maintains sorted data.

Operations of B+ Trees:

- The operations supported in B+ trees are Insertion, deletion and searching with the time complexity of $O(\log n)$ for every operation.
- They are almost similar to the B tree operations as the base idea to store data in both data structures is same.
- However, the difference occurs as the data is stored only in the leaf nodes of a B+ trees, unlike B trees.

Insertion operation:

- The insertion to a B+ tree starts at a leaf node.
- Step 1 – Calculate the maximum and minimum number of keys to be added onto the B+ tree node.



Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4

Order = 4

Maximum Children (m) = 4

Minimum Children ($\lceil \frac{m}{2} \rceil$) = 2

Maximum Keys ($m - 1$) = 3

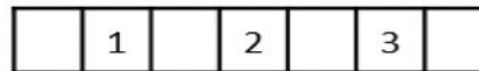
Minimum Keys ($\lceil \frac{m-1}{2} \rceil$) = 1

Insertion operation:

- Step 2 – Insert the elements one by one accordingly into a leaf node until it exceeds the maximum key number.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4

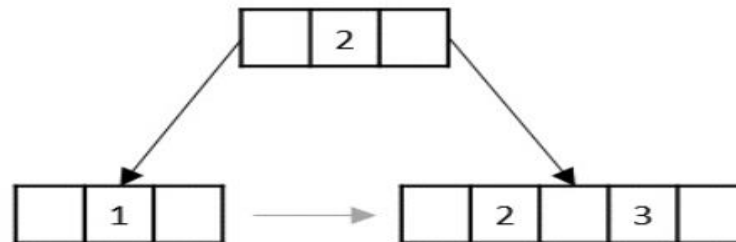
Adding 4 into
this node will
lead to an
overflow



Insertion operation:

- Step 3 – The node is split into half where the left child consists of minimum number of keys and the remaining keys are stored in the right child.

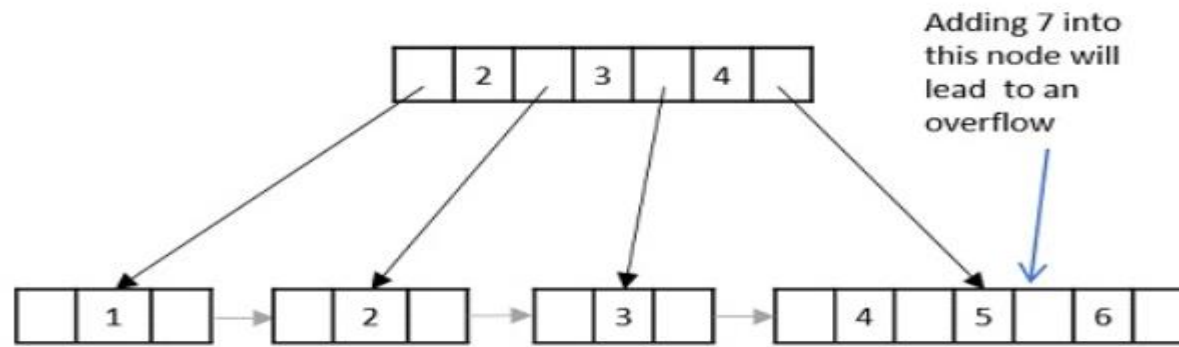
Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



Insertion operation:

- Step 4 – But if the internal node also exceeds the maximum key property, the node is split in half where the left child consists of the minimum keys and remaining keys are stored in the right child.
- However, the smallest number in the right child is made the parent.

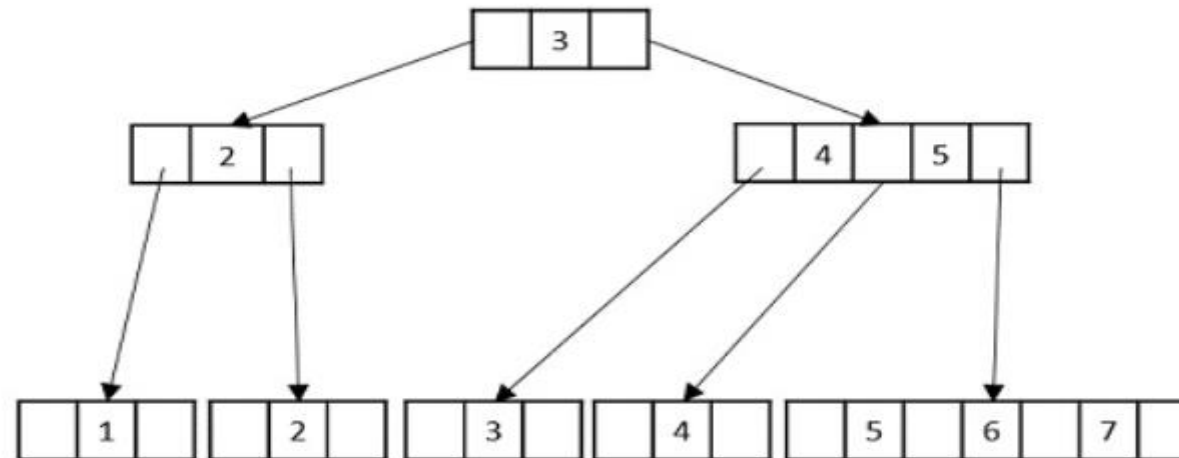
Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



Insertion operation:

- Step 5 – If both the leaf node and internal node have the maximum keys, both of them are split in the similar manner and the smallest key in the right child is added to the parent node.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



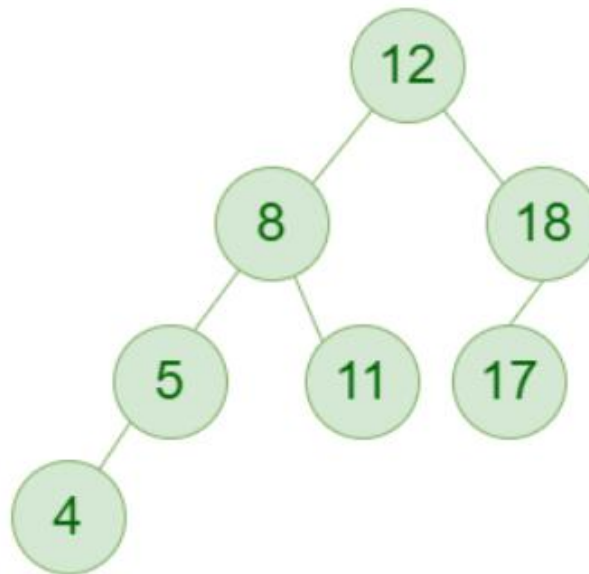
B tree vs B+ tree:

Basis of Comparison	B tree	B+ tree
Pointers	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
Search	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and more accurate.
Redundant Keys	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at the leaf.
Insertion	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Deletion	Deletion of the internal node is very complex and the tree has to undergo a lot of transformations.	Deletion of any node is easy because all node are found at leaf.
Leaf Nodes	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.

AVL Tree:

- An AVL tree defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left and right sub-trees for any node cannot be more than one.
- The difference between the heights of the left sub-tree and the right sub-tree for any node is known as the balance factor of the node.
- The AVL tree is named after its inventors, Georgy Adelson-Velsky and Evgenii Landis, who published it in their 1962 paper "An algorithm for the organization of information".

Representation of AVL Tree:

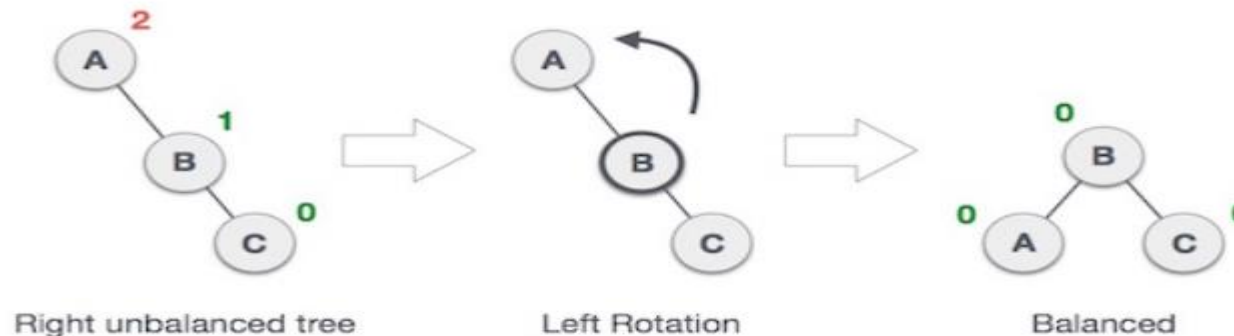


Operations on an AVL Tree:

- Insertion, Deletion, Searching.
- Rotating the sub-trees in an AVL Tree:
- An AVL tree may rotate in one of the following four ways to keep itself balanced:

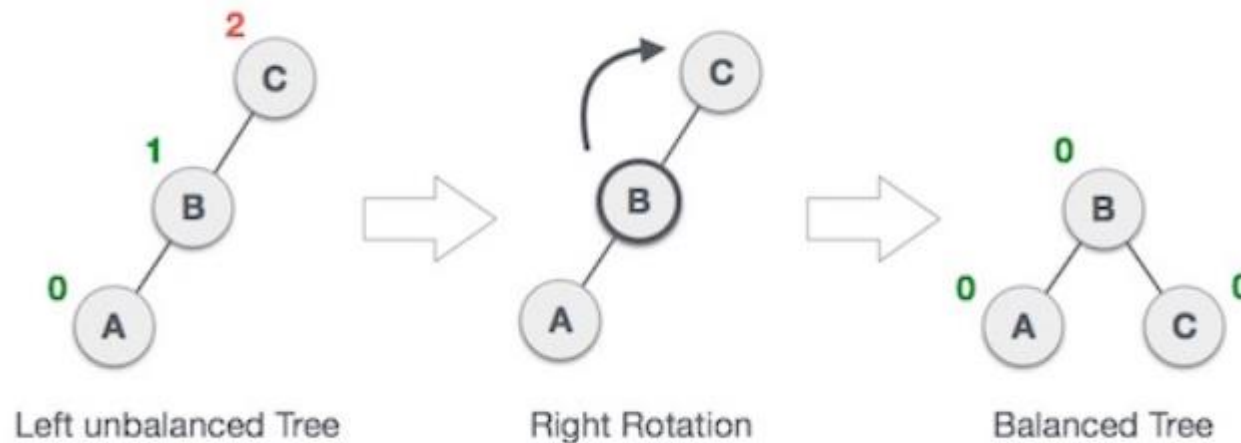
Left Rotation:

- When a node is added into the right sub-tree of the right sub-tree, if the tree gets out of balance, we do a single left rotation.



Right Rotation:

- If a node is added to the left sub-tree of the left sub-tree, the AVL tree may get out of balance, we do a single right rotation.



Insertion operation:

- The data is inserted into the AVL Tree by following the Binary Search Tree property of insertion, the left sub-tree must contain elements less than the root value and right sub-tree must contain all the greater elements.
- However, in AVL Trees, after the insertion of each element, the balance factor of the tree is checked; if it does not exceed 1, the tree is left as it is.
- But if the balance factor exceeds 1, a balancing algorithm is applied to readjust the tree such that balance factor becomes less than or equal to 1 again.

Algorithm:

- Step 1 – Create a node
- Step 2 – Check if the tree is empty
- Step 3 – If the tree is empty, the new node created will become the root node of the AVL Tree.
- Step 4 – If the tree is not empty, we perform the Binary Search Tree insertion operation and check the balancing factor of the node in the tree.
- Step 5 – Suppose the balancing factor exceeds ± 1 , we apply suitable rotations on the said node and resume the insertion from Step 4.

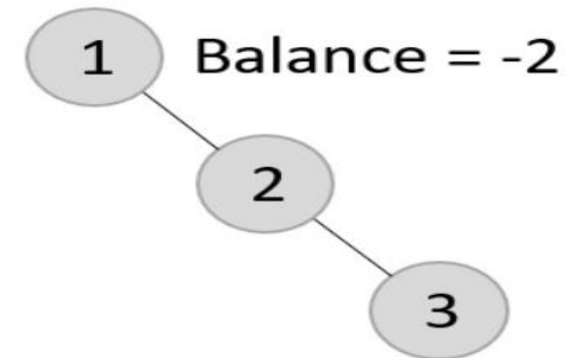
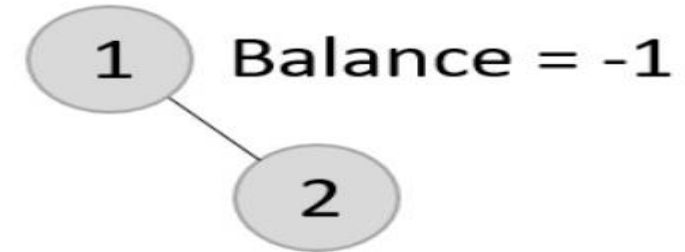
Insertion operation:

- Let us understand the insertion operation by constructing an example AVL tree with 1 to 7 integers.
- Starting with the first element 1, we create a node and measure the balance, 0.

1 Balance = 0

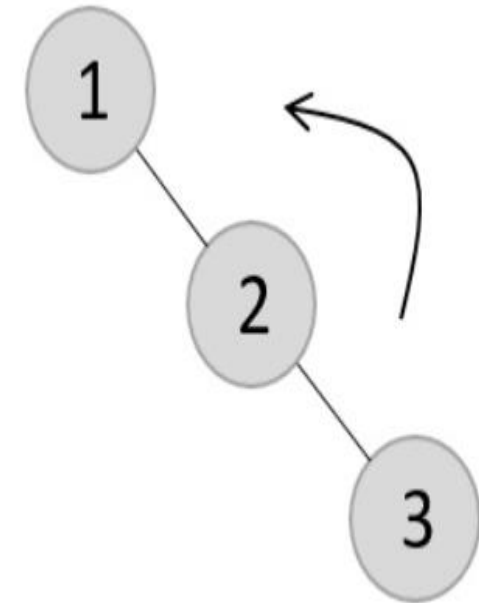
Insertion operation:

- Since both the binary search property and the balance factor are satisfied, we insert another element into the tree.
- The balance factor for the two nodes are calculated and is found to be -1 (Height of left sub-tree is 0 and height of the right sub-tree is 1).
- Since it does not exceed 1, we add another element to the tree.



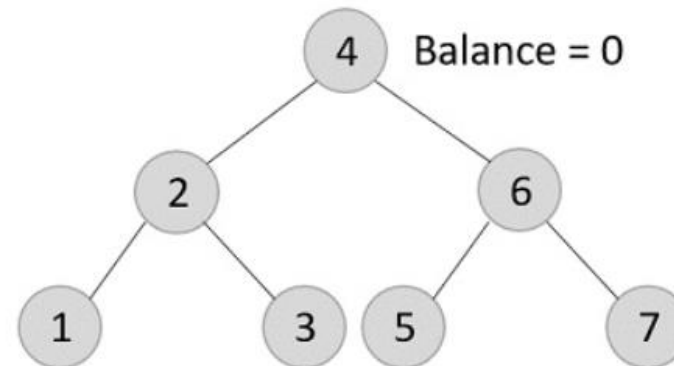
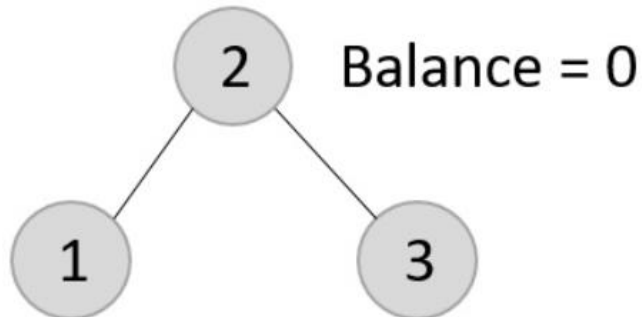
Insertion operation:

- Now, after adding the third element, the balance factor exceeds 1 and becomes 2. Therefore, rotations are applied.
- In this case, the RR rotation is applied since the imbalance occurs at two right nodes.



Insertion operation:

- The tree is rearranged as:
- Similarly, the next elements are inserted and rearranged using these rotations.
- After rearrangement, we achieve the tree as:

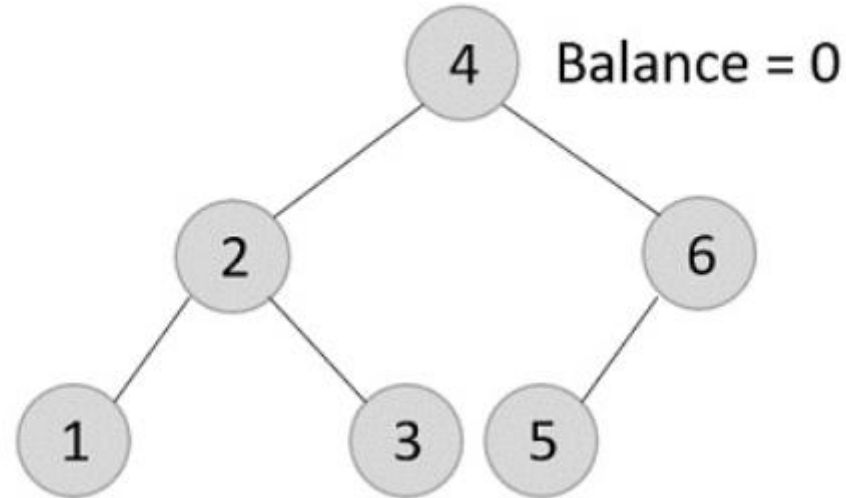


Deletion operation:

- Deletion in the AVL Trees take place in three different scenarios –
- **Scenario 1** (Deletion of a leaf node) – If the node to be deleted is a leaf node, then it is deleted without any replacement as it does not disturb the binary search tree property. However, the balance factor may get disturbed, so rotations are applied to restore it.
- **Scenario 2** (Deletion of a node with one child) – If the node to be deleted has one child, replace the value in that node with the value in its child node. Then delete the child node. If the balance factor is disturbed, rotations are applied.
- **Scenario 3** (Deletion of a node with two child nodes) – If the node to be deleted has two child nodes, find the in-order successor of that node and replace its value with the in-order successor value. Then try to delete the in-order successor node. If the balance factor exceeds 1 after deletion, apply balance algorithms.

Deletion operation:

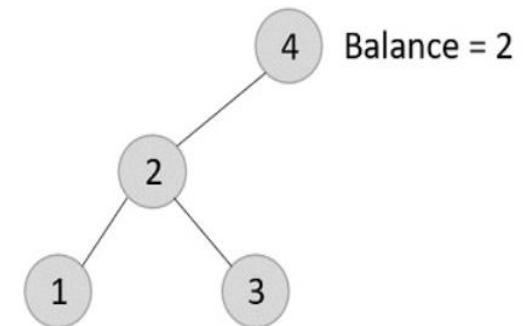
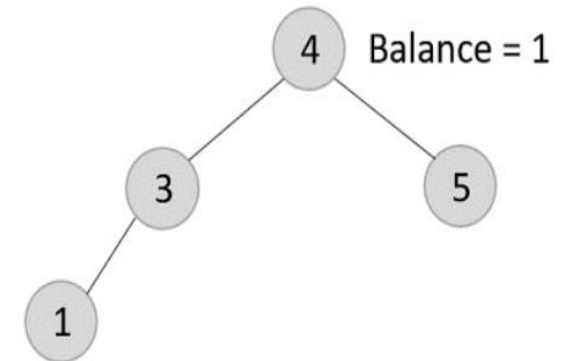
- Using the same tree given above, let us perform deletion in three scenarios



- Deleting element 7 from the tree above

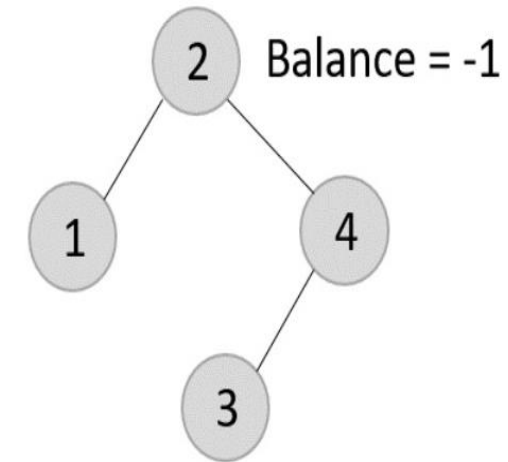
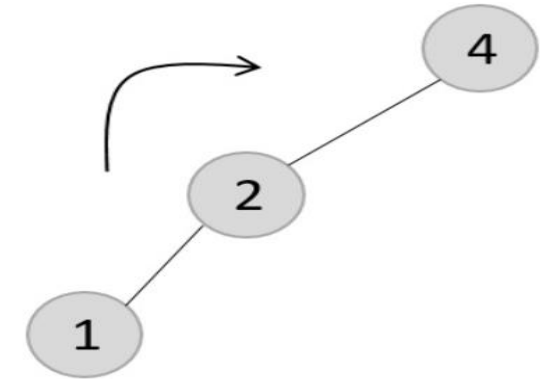
Deletion operation:

- Since the element 7 is a leaf, we normally remove the element without disturbing any other node in the tree.
- The balance of the tree becomes 1, and since it does not exceed 1 the tree is left as it is.
- If we delete the element 5 further, we would have to apply the left rotations; either LL or LR since the imbalance occurs at both 1-2-4 and 3-2-4.



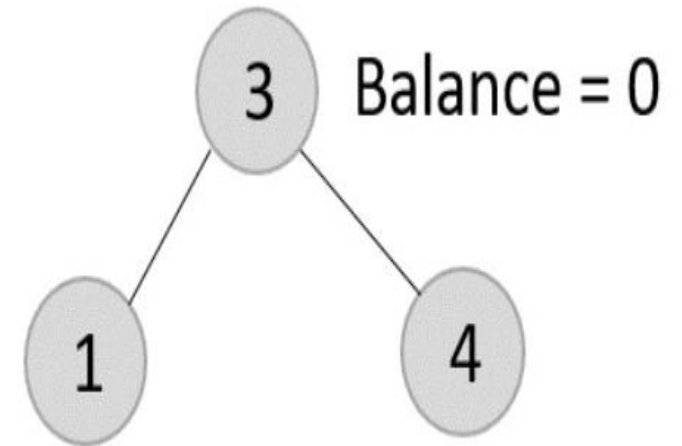
Deletion operation:

- The balance factor is disturbed after deleting the element 5, therefore we apply LL rotation (we can also apply the LR rotation here).
- Once the LL rotation is applied on path 1-2-4, the node 3 remains as it was supposed to be the right child of node 2 (which is now occupied by node 4).
- Hence, the node is added to the right subtree of the node 2 and as the left child of the node 4.



Deletion operation:

- Deleting element 2 from the remaining tree
- As mentioned in scenario 3, this node has two children.
- Therefore, we find its in-order successor that is a leaf node (say, 3) and replace its value with the in-order successor.
- The balance of the tree still remains 1, therefore we leave the tree as it is without performing any rotations.



Splay Tree:

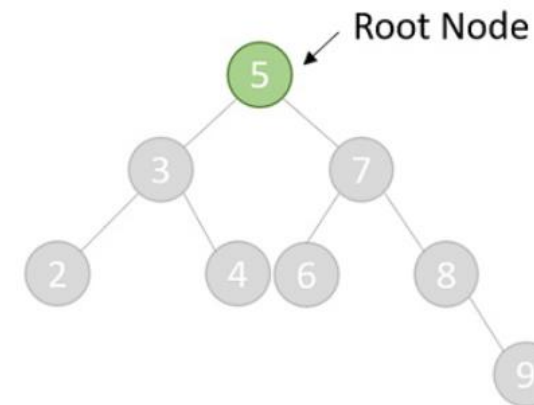
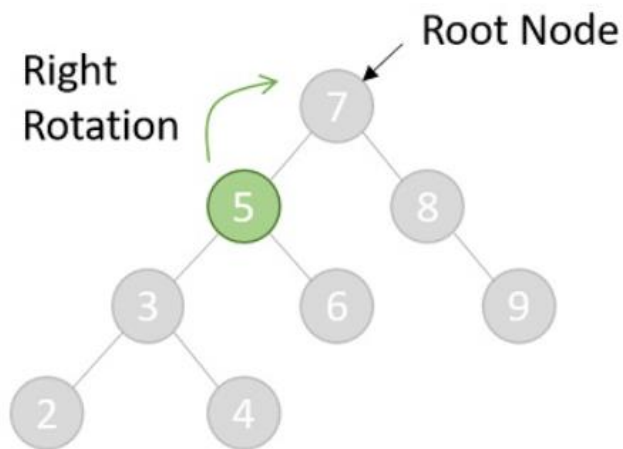
- Splay trees are the altered versions of the Binary Search Trees, since it contains all the operations of BSTs, like insertion, deletion and searching, followed by another extended operation called splaying.
- For instance, a value "A" is supposed to be inserted into the tree. If the tree is empty, add "A" to the root of the tree and exit; but if the tree is not empty, use binary search insertion operation to insert the element and then perform splaying on the new node.
- Similarly, after searching an element in the splay tree, the node consisting of the element must be splayed as well.

Types of rotations:

- There are six types of rotations for it.
- Zig rotation
- Zag rotation
- Zig-Zig rotation
- Zag-Zag rotation
- Zig-Zag rotation
- Zag-Zig rotation

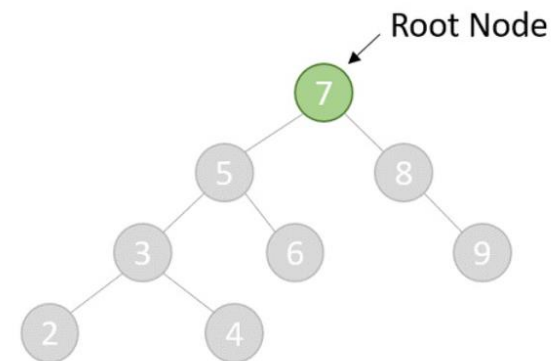
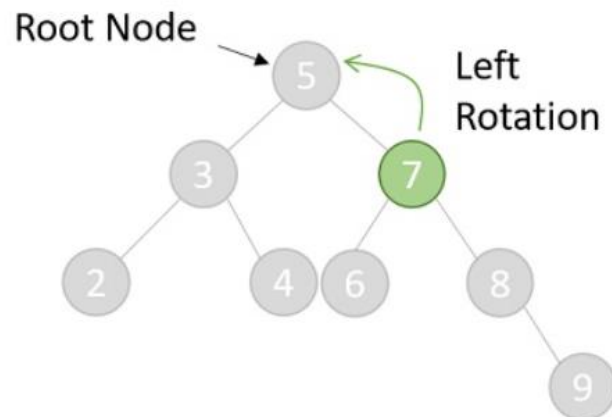
Zig rotation:

- The zig rotations are performed when the operational node is either the root node or the left child node of the root node.
- The node is rotated towards its right.
- After the shift, the tree will look like



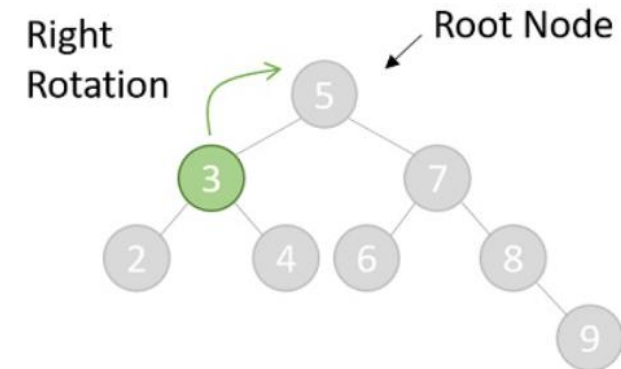
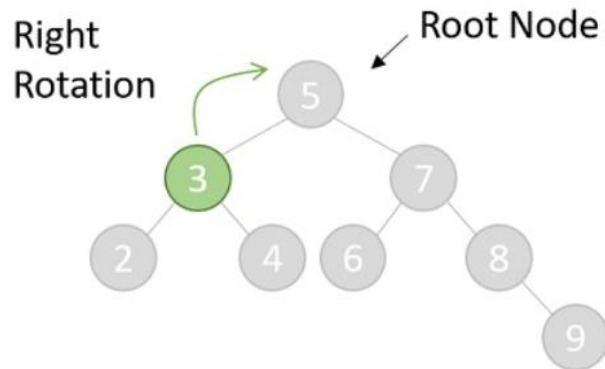
Zag rotation:

- The zag rotations are also performed when the operational node is either the root node or the right child node of the root node.
- The node is rotated towards its left.
- The operational node becomes the root node after the shift



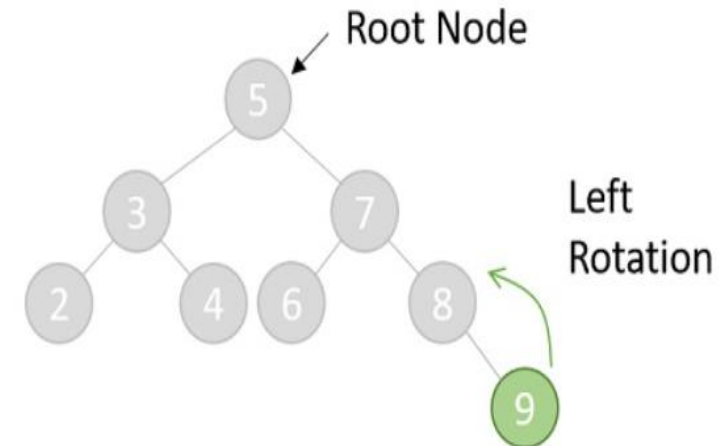
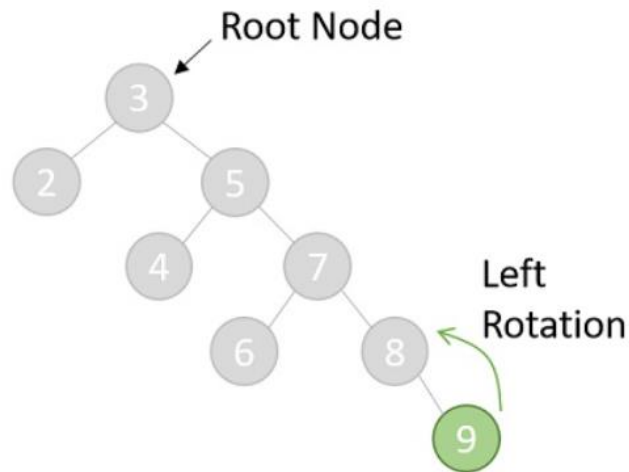
Zig-Zig rotation

- The zig-zig rotations are performed when the operational node has both parent and a grandparent. The node is rotated two places towards its right.
- The first rotation will shift the tree to one position right



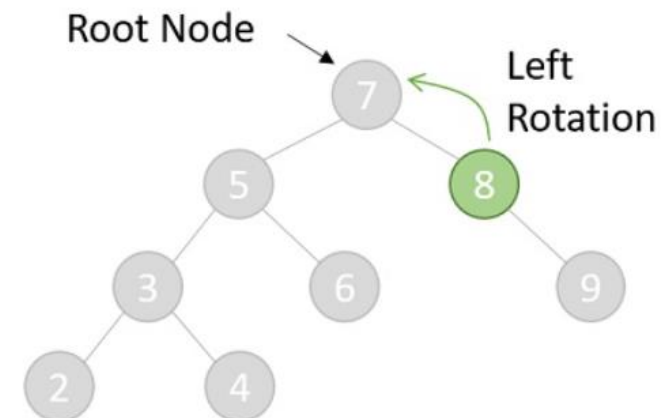
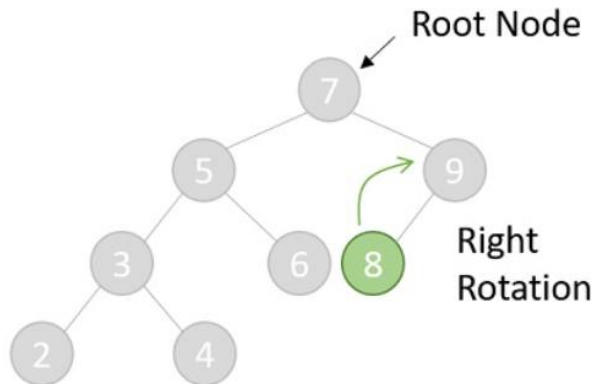
Zag-Zag rotation:

- The zag-zag rotations are also performed when the operational node has both parent and a grandparent.
- The node is rotated two places towards its left.
- After the first rotation, the tree will look like –



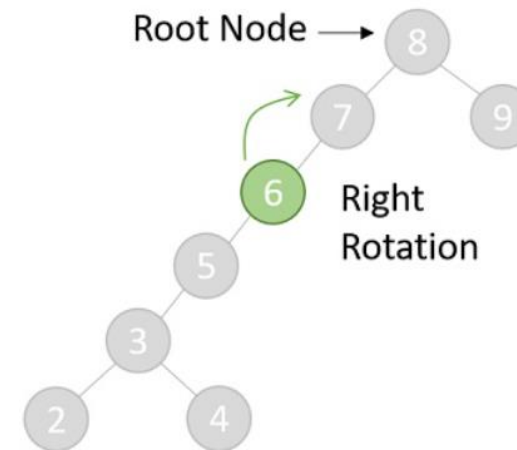
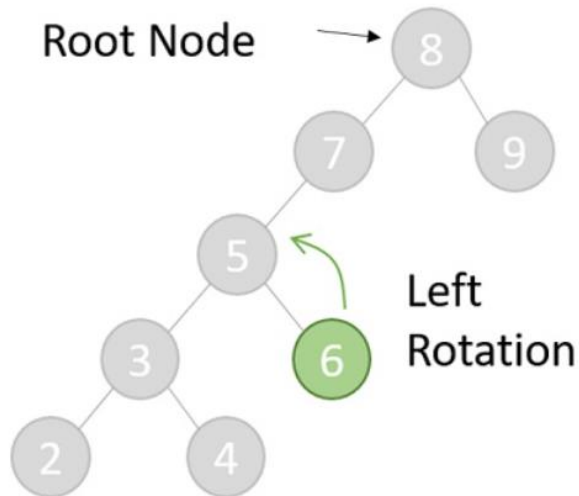
Zig-Zag rotation:

- The zig-zag rotations are performed when the operational node has both a parent and a grandparent.
- But the difference is the grandparent, parent and child are in LRL format. The node is rotated first towards its right followed by left.



Zag-Zig rotation:

- The zag-zig rotations are also performed when the operational node has both parent and grandparent.
- But the difference is the grandparent, parent and child are in RLR format. The node is rotated first towards its left followed by right



Insertion operation:

- The insertion operation in a Splay tree is performed in the exact same way insertion in a binary search tree is performed.
- The procedure to perform the insertion in a splay tree is given as follows Check whether the tree is empty; if yes, add the new node and exit

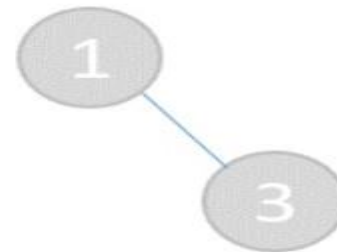
Insert **1, 3** into the Splay Tree



Insertion operation:

- If the tree is not empty, add the new node to the existing tree using the binary search insertion.

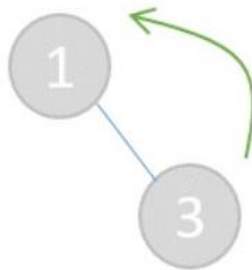
Insert **1**, **3** into the Splay Tree



Insertion operation:

- Then, suitable splaying is chosen and applied on the newly added node.

Insert **1**, **3** into the Splay Tree



Insert **1**, **3** into the Splay Tree

